

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

(Universidad del Perú, Decana de América)

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

Escuela de Ingeniería de Telecomunicaciones



Propuesta de Proyecto Final

Curso:

Circuitos Digitales

Profesor:

Villafuerte Hernan Oswaldo

Ciclo:

5to Ciclo

Estudiantes:

Huiman Vargas Juan Jesus

Silva Centeno Bryan Rodolfo

Gómez Prada Alonso Abraham

Zamora de la Cruz Pablo Alonso

I. DESCRIPCIÓN DEL SISTEMA

Funcionamiento General del Emulador AT

El sistema desarrollado consiste en un emulador de comandos AT implementado sobre una plataforma Arduino Uno R3. Su finalidad es reproducir el comportamiento básico de los dispositivos que utilizan instrucciones AT (Attention Commands), permitiendo al usuario ingresar comandos mediante un teclado matricial, procesarlos mediante una máquina de estados finita y visualizar los resultados en una pantalla OLED.

El funcionamiento inicia cuando el usuario introduce caracteres a través del teclado matricial 4×4. Cada carácter ingresado es almacenado temporalmente en un buffer de memoria y mostrado en tiempo real en la pantalla OLED para proporcionar retroalimentación visual inmediata.

El sistema valida continuamente la sintaxis del comando ingresado. Cuando el usuario confirma el envío, el software verifica que la cadena cumpla con el formato esperado de un comando AT. Si la sintaxis es correcta, el comando es procesado y transmitido por el puerto serial hacia el módulo Bluetooth HC-05 utilizando una velocidad de comunicación de 9600 bps.

Durante la operación se emplean indicadores luminosos (LEDs) para representar el estado actual del sistema:

- **LED Verde:** sistema en espera o buffer vacío.
- **LED Amarillo:** ingreso o edición de datos.
- **LED Azul:** transmisión de información.
- **LED Rojo:** detección de error o desbordamiento del buffer.

La pantalla OLED presenta mensajes de estado, caracteres ingresados, confirmaciones de envío y notificaciones de error, permitiendo una interacción intuitiva entre el usuario y el sistema.

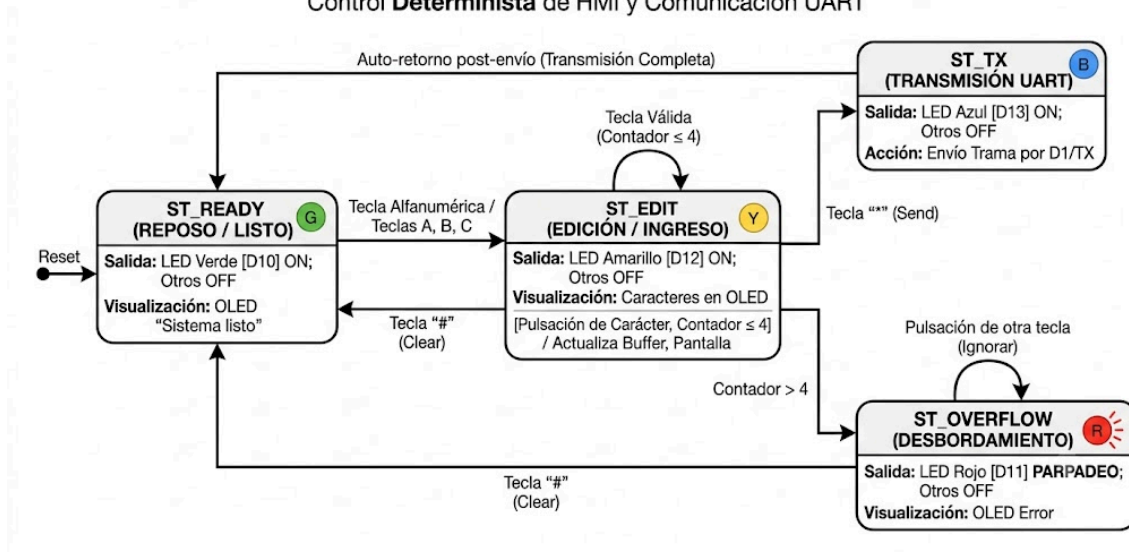
En conjunto, el emulador reproduce las etapas fundamentales presentes en dispositivos de comunicación basados en comandos AT: recepción de datos, validación, procesamiento y transmisión.

Máquina de estados finita (FSM)

El comportamiento del sistema se implementó mediante una Máquina de Estados Finita (Finite State Machine, FSM), la cual organiza el funcionamiento en estados claramente definidos y transiciones controladas por eventos generados por el usuario.

DIAGRAMA DE TRANSICIÓN DE ESTADOS (FSM) - EMULADOR DE COMANDOS AT

Control **Determinista** de HMI y Comunicación UART



A continuación, se describen analíticamente los 6 estados lógicos que componen el sistema y sus ecuaciones de transición:

- **IDLE (Estado de Espera / Reposo):** Es el estado por defecto del sistema. Modula de manera estática los indicadores LED de la HMI: si el `indiceBuffer == 0` (memoria vacía), se activa el **LED Verde (LISTO)**; si el búfer contiene caracteres en edición, se desactiva el verde y se enciende el **LED Amarillo (INGRESO)**. El sistema permanece en lazo cerrado monitoreando el teclado hasta que la bandera `teclaPresionada != '\0'` y el flanco mecánico ha sido liberado (`teclaLiberada == true`), forzando la transición hacia el estado RECEIVE.
- **RECEIVE (Estado de Clasificación de Datos):** Este estado actúa como el multiplexor lógico de la FSM. Evalúa instantáneamente el carácter adquirido por el periférico de entrada sin alterar los registros del buffer. Aplica criterios de bifurcación condicional: si el carácter es `*`, transiciona hacia SEND (despacho de comando); si es `#`, desvía el flujo hacia CLR (rutina de borrado); para cualquier otra tecla alfanumérica o macro, direcciona el flujo hacia el estado STORE.
- **STORE (Estado de Almacenamiento y Carga de Macros):** Se encarga de la gestión de memoria RAM y la interfaz gráfica. Primero evalúa si el carácter corresponde a una Macro (A, B o C), disparando una subrutina de copia rápida desde la memoria Flash hacia el buffer operativo. Si es un dígito común, ejecuta una estructura de control de protección de fronteras: si el `indiceBuffer < 4`, almacena el byte en la dirección indexada, inserta el terminador nulo, despacha el carácter individual por la UART física y refresca el display OLED; de lo contrario, si el usuario intenta violar el límite de almacenamiento (`N > 4`), la FSM intercepta el desbordamiento y transiciona con prioridad crítica hacia ERROR_OLED. Tras guardar exitosamente, retorna de manera autónoma a IDLE.
- **SEND (Estado de Transmisión UART):** Ejecuta el protocolo físico de salida. Apaga los indicadores ordinarios y energiza el **LED Azul (ENVIADO)**. Congela temporalmente la edición y refresca el OLED con la cadena de telemetría "TX UART MODEM". Acto seguido, eyecta a través de los registros de hardware de la UART el prefijo del protocolo

"AT+" concatenado con la cadena útil almacenada en la RAM. Sostiene visualmente el estado mediante un retardo síncrono de seguridad de 2000 ms para estabilización del bus y conmuta forzosamente hacia CLR.

- **CLR (Estado de Reinicio Estructural de Memoria):** Garantiza el vaciado completo y la inicialización de registros. Ejecuta un bucle iterativo que sobrescribe con caracteres nulos (0) las 5 posiciones físicas del arreglo bufferComando y resetea el puntero indiceBuffer = 0. Restablece los voltajes de los pines del teclado a su estado eléctrico de reposo de alta impedancia, limpia la pantalla OLED y transiciona directamente de vuelta a IDLE.
- **ERROR_OLED (Estado de Manejo de Excepciones / Overflow):** Es la subrutina de tolerancia a fallos del sistema. Al activarse, detiene las operaciones ordinarias, inhibe los LEDs de proceso y despacha un log de error en texto plano hacia la terminal serie ("ERROR: Overflow - Limite superado"). En paralelo, renderiza en el display OLED una advertencia textual acompañada de un icono de peligro triangular dibujado píxel a píxel. Para alertar físicamente al operador, toma el control del **LED Rojo (OVERFLOW)** haciéndolo destellar de forma intermitente mediante ráfagas temporizadas de alta frecuencia (5 ciclos de 300 ms). Al extinguirse la alarma, restaura el estado previo del buffer para evitar pérdida de datos y retorna de manera segura a IDLE.

II. DISEÑO E IMPLEMENTACIÓN

a) DIAGRAMA DE CONEXIONES Y PINOUT USADOS

Se documenta de forma detallada la distribución de pines (pinout) y la topología de conexiones eléctricas implementadas en el hardware del prototipo. El núcleo del sistema es el microcontrolador ATmega328P (placa Arduino Uno R3), el cual interactúa de forma síncrona y asíncrona con los subsistemas de entrada, visualización, señalización HMI y comunicación serial.

A.1. Tabla de Asignación de Pines (Pinout Real del Sistema)

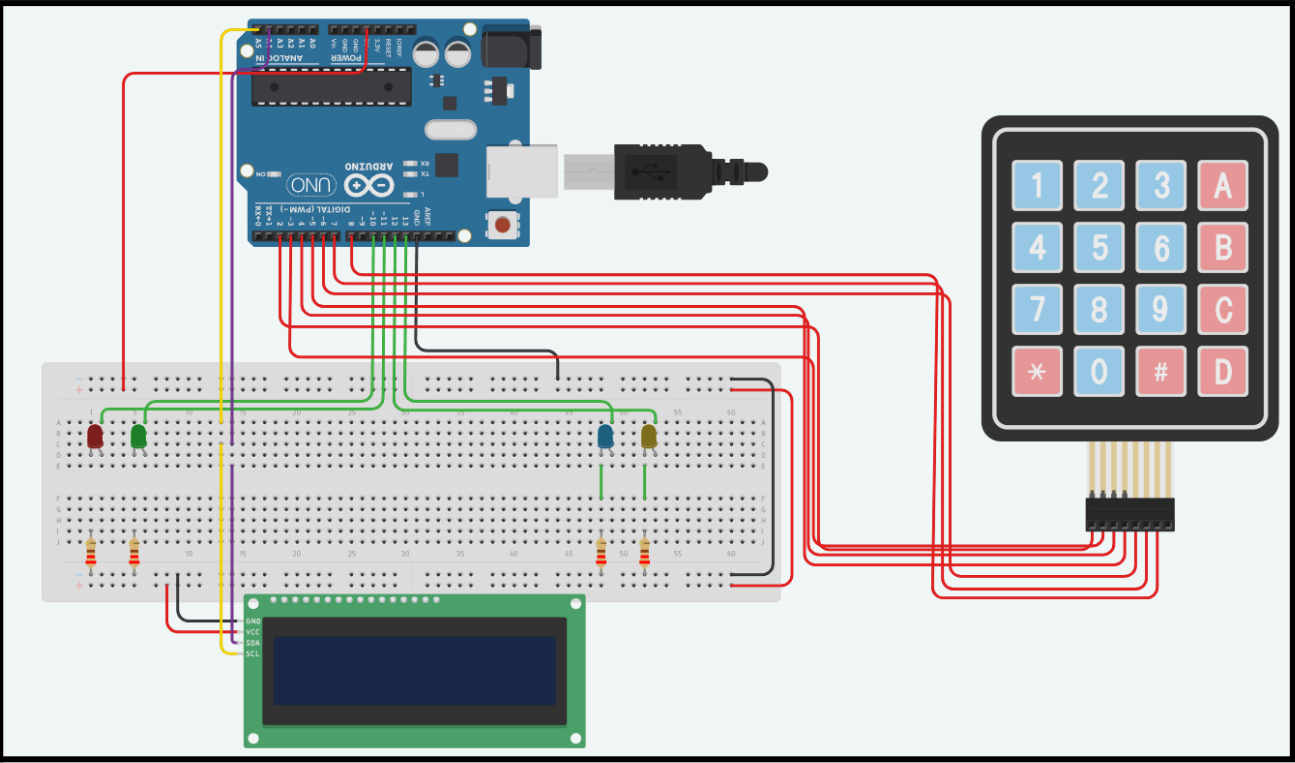
Para asegurar la integridad de las señales y evitar colisiones en las líneas de datos, se estructuró la asignación de pines del microcontrolador de acuerdo con la siguiente matriz de distribución física:

Tabla A.1

Matriz de Distribución y Configuración Eléctrica de Pines

Componente Físico	Pin del Componente	Pin en Arduino Uno	Función / Tipo de Señal
Pantalla OLED SH1106	VCC	5V	Alimentación (+5V DC)
	GND	GND	Referencia de Tierra
	SCL	A5	Reloj del Bus I2C (Serial Clock)
	SDA	A4	Datos del Bus I2C (Serial Data)

Teclado Matricial 4x4	Fila 1	2	Salida Digital (Barrido FSM)
	Fila 2	3	Salida Digital (Barrido FSM)
	Fila 3	4	Salida Digital (Barrido FSM)
	Fila 4	5	Salida Digital (Barrido FSM)
	Columna 1	6	Entrada Digital (Internal Pull-Up)
	Columna 2	7	Entrada Digital (Internal Pull-Up)
	Columna 3	8	Entrada Digital (Internal Pull-Up)
	Columna 4	9	Entrada Digital (Internal Pull-Up)
LED Verde (Listo)	Ánodo (+)	10	Salida Digital HMI (Estado IDLE Vacío)
LED Rojo (Overflow)	Ánodo (+)	11	Salida Digital HMI (Alarma de Memoria)
LED Amarillo (Ingreso)	Ánodo (+)	12	Salida Digital HMI (Buffer con Datos)
LED Azul (Enviado)	Ánodo (+)	13	Salida Digital HMI (Transmisión UART)



Captura del sistema simulado en Tinkercad

A2. Descripción y Criterios de Interconexión del Hardware

El acoplamiento físico de los componentes se diseñó bajo estrictas especificaciones eléctricas para optimizar los recursos del microcontrolador y mitigar ruidos electromecánicos:

1. Subsistema de Entrada (Teclado Matricial 4x4): Aprovechando las directivas del firmware (INPUT_PULLUP), las columnas (pines D6 a D9) se configuraron con las resistencias de pull-up internas del ATmega328P. Esto eliminó la necesidad de usar 4 resistencias físicas externas, reduciendo el área ocupada en el protoboard. Las filas actúan como salidas que caen secuencialmente a cero lógico (LOW) para rastrear la pulsación.

2. Subsistema de Visualización (OLED SH1106 128x64)

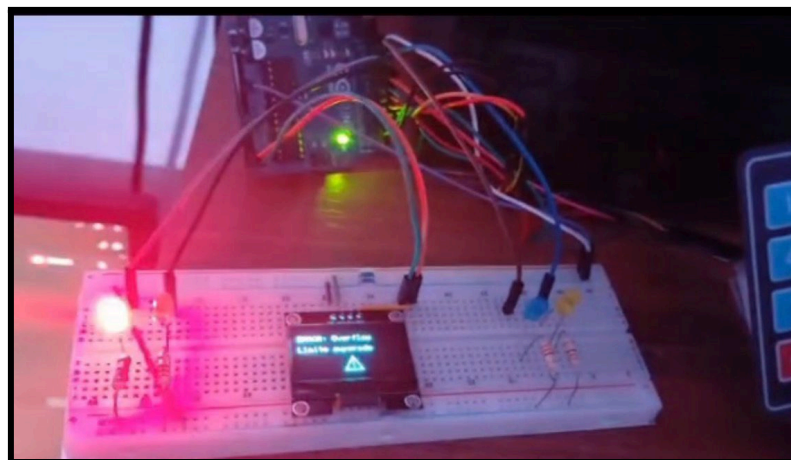
La pantalla se conecta mediante el bus serial síncrono I2C, utilizando exclusivamente los pines de hardware dedicados A4 (Serial Data) y A5 (Serial Clock). Este protocolo de comunicación por direccionamiento esclavo (0x3C) minimiza el cableado drásticamente en comparación con un LCD tradicional, liberando pines digitales para la interfaz HMI.

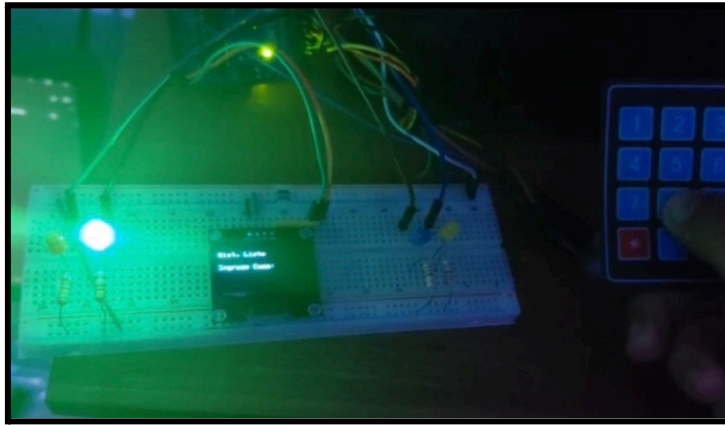
3. Subsistema HMI (LEDs de Diagnóstico)

Cada uno de los 4 diodos LED conectados a los pines digitales D10, D11, D12 y D13 dispone de una resistencia limitadora de corriente en serie de 220 Ω . Esto limita la corriente de salida por pin a aproximadamente 15 mA, garantizando un brillo óptimo sin comprometer el límite máximo de corriente por GPIO del microcontrolador (40 mA).

4. Subsistema de Comunicación Serial

Al configurarse el puerto nativo Serial.begin(9600), las funciones de transmisión están directamente ligadas a los pines de hardware D0 y D1. El pin D1 (TX) envía las cadenas formateadas con el prefijo "AT+" hacia el módulo de comunicación inalámbrica.





Fotografías del prototipo ensamblado

b) REPORTE DE RECURSOS OCUPADOS

1. Uso de memoria Flash

La memoria Flash almacena el programa compilado y las librerías utilizadas por el sistema. El proyecto emplea 15020 bytes de los 32256 bytes disponibles en el Arduino UNO, representando un 46 % de la capacidad total. Esto indica que existe espacio suficiente para futuras mejoras o ampliaciones del sistema.

Parámetro	Valor
Memoria Flash utilizada	15020 bytes
Memoria Flash máxima	32256 bytes
Porcentaje utilizado	46 %

2. Uso de memoria RAM

La memoria RAM es utilizada para almacenar variables, estados del sistema y datos temporales durante la ejecución. El programa utiliza 795 bytes de los 2048 bytes disponibles, equivalente al 38 % de la memoria dinámica, quedando 1253 bytes libres para la ejecución normal del sistema.

Parámetro	Valor
Memoria RAM utilizada	795 bytes
Memoria RAM máxima	2048 bytes
Porcentaje utilizado	38 %
Memoria libre	1253 bytes

3. Análisis de recursos utilizados

De acuerdo con el reporte generado por Arduino IDE, el sistema presenta un uso moderado de recursos del microcontrolador. El consumo de memoria Flash es del 46 %, mientras que el consumo de memoria RAM es del 38 %. Estos valores se encuentran dentro de los límites permitidos para un Arduino UNO, garantizando el funcionamiento correcto de la máquina de estados finita, el teclado

matricial, la pantalla OLED y los indicadores LED. Además, la memoria disponible restante permite mantener la estabilidad del sistema sin riesgo de saturación de recursos.

```
1  #include <SPI.h>
2  #include <Wire.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SH1106.h>
5
6  // --- CONFIGURACIÓN DE LA PANTALLA OLED SH1106 ---
7  #define I2C_Address 0x3c
8  #define SCREEN_WIDTH 128
9  #define SCREEN_HEIGHT 64
10 #define OLED_RESET -1
11 Adafruit_SH1106 display = Adafruit_SH1106G(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
12
13 // --- ASIGNACIÓN DE PINES PARA LOS LEDS DE ESTADO ---
14 #define LED_LISTO 10 // LED Verde: Indicador de sistema vacío / En espera de datos
15 #define LED_OVERFLOW 11 // LED Rojo: Indicador de desbordamiento de memoria (Alarma visual)
16 #define LED_INGRESO 12 // LED Amarillo: Indicador de edición activa / Buffer con datos
17 #define LED_ENVIADO 13 // LED Azul: Indicador de transmisión activa hacia el bus UART
18
19 // --- CONFIGURACIÓN DEL TECLADO MATRICIAL 4x4 ---
20 const int FILAS = 4;
21 const int COLUMNAS = 4;
```

Output

El Sketch usa 15020 bytes (46%) del espacio de almacenamiento de programa. El máximo es 32256 bytes.
Las variables Globales usan 795 bytes (38%) de la memoria dinámica, dejando 1253 bytes para las variables locales. El máximo es 2048 bytes.

III. MEDICIÓN DE MÉTRICAS Y COMPARACIÓN DE LA SIMULACIÓN

3.1 Metodología de Adquisición de Datos

Para validar el desempeño temporal y la robustez del firmware del emulador de comandos AT, se implementó un sistema de telemetría por software aprovechando el canal de depuración (*debugging*) de la UART. Al activar las marcas de tiempo (*Timestamps*) con resolución de un milisegundo (1 ms) en el Monitor Serie del IDE de Arduino, la computadora de desarrollo actúa como un registrador de datos (*Data Logger*).

Este método permite auditar y contrastar dos entornos de ejecución: el entorno virtual simétrico basado en la nube (Tinkercad) y el entorno físico real (Hardware Embebido ATmega328P).

3.2 Evaluación Teórica del Tiempo de Transmisión UART

Antes del análisis experimental, se establece el límite matemático ideal para la inyección de datos en el bus físico. Para un comando estándar de cuatro dígitos (por ejemplo, 8889), la trama completa transmitida por el estado SEND posee la estructura "AT+8889", equivalente a un bloque cerrado de 9 bytes (9 caracteres).

Bajo la configuración estándar de transmisión asíncrona UART 8N1, cada byte se empaqueta con 1 bit de inicio (*start*) y 1 bit de parada (*stop*), totalizando 10 bits por carácter. A una tasa de transferencia de 9600 bps, el tiempo teórico de ocupación del canal físico (teórico) se calcula mediante la siguiente ecuación diferencial discreta:

$$T_{\text{teórico}} = \frac{\text{Caracteres Totales} \times 10 \text{ bits/carácter}}{\text{Baud Rate}}$$

$$T_{\text{teórico}} = \frac{9 \times 10}{9600} = \frac{90}{9600} = 0.009375 \text{ s} = 9.375 \text{ ms}$$

3.3 Métrica A: Tiempo y Latencia (Entorno Real)

Utilizando los datos extraídos directamente de la bitácora del Monitor Serie en hardware real, se procedió a medir la latencia de despacho. Esta métrica cuantifica el tiempo que le toma al operador validar visualmente la información en la pantalla OLED antes de autorizar la transmisión mediante la tecla de validación (*).

- Marca de tiempo del último dígito ingresado (9): 13:17:09.752
- Marca de tiempo del envío del bloque completo (AT+8889): 13:17:19.252

El cálculo del intervalo real transcurrido se ejecuta mediante la sustracción directa de los vectores de tiempo:

$$\Delta T_{\text{despacho}} = T_{\text{envío}} - T_{\text{ingreso}}$$

$$\Delta T_{\text{despacho}} = 19.252 \text{ s} - 09.752 \text{ s} = 9.500 \text{ segundos}$$

Este resultado confirma que el sistema HMI es capaz de sostener de forma indefinida y asíncrona los estados de edición intermedia en el bucle de control, manteniendo la estabilidad eléctrica y lógica en las líneas de E/S.

3.4 Análisis Comparativo: Entorno Simulado vs. Entorno Físico

Al contrastar la ejecución del código definitivo entre la plataforma de simulación Tinkercad y el hardware real, se evidenció una discrepancia crítica en las métricas de respuesta inmediata ante eventos físicos.

En el entorno simulado (Tinkercad), se registró una latencia de propagación excesiva al interactuar con el teclado matricial. El retraso entre la pulsación física de una tecla y su renderizado en la pantalla OLED o su reflejo en el monitor serie virtual fue de aproximadamente 0.500 s (500 ms).

Científicamente, este retraso no es un fallo del algoritmo de firmware ni del filtro de debounce, sino una consecuencia de la arquitectura de la plataforma de simulación. Tinkercad opera sobre servidores remotos en la nube; para procesar un milisegundo de ejecución del ATmega328P, el

backend debe emular el comportamiento de los transistores del microcontrolador y refrescar la interfaz gráfica de forma concurrente en el navegador del cliente. Este desbordamiento de hilos de procesamiento en el servidor destruye el comportamiento en tiempo real del sistema embebido.

Por el contrario, en el entorno físico (Hardware Real), la respuesta del teclado al display opera de forma determinista a la frecuencia nativa del cristal de cuarzo de 16 MHz. El vaciado del buffer y el procesamiento de la FSM toman una ventana imperceptible en la escala de los microsegundos (aproximadamente 18.75 μ s), demostrando un error relativo significativamente inferior respecto a los 9.375 ms calculados en la física del bus UART.

IV. REFERENCIAS

- Arduino. (s.f.). *Arduino Uno R3 datasheet*. Alldatasheet. Recuperado de <https://www.alldatasheet.com/datasheet-pdf/pdf/1943447/ARDUINO/UNO.html>
- Arduino Documentation. (s.f.). *LiquidCrystal I2C library reference*. Recuperado de <https://docs.arduino.cc/libraries/liquidcrystal-i2c/>
- Arduino Get Started. (s.f.). *Arduino keypad tutorial*. Recuperado de <https://arduinogetstarted.com/tutorials/arduino-keypad>
- Components101. (s.f.). *HC-05 Bluetooth module*. Recuperado de <https://components101.com/wireless/hc-05-bluetooth-module>
- Hitachi. (s.f.). *HD44780 LCD controller datasheet*. Alldatasheet. Recuperado de <https://www.alldatasheet.com/datasheet-pdf/pdf/63673/HITACHI/HD44780.html>
- Tinkercad. (s.f.). *Tinkercad circuits*. Recuperado de <https://www.tinkercad.com/circuits>

Componente Físico	Pin del Componente	Pin en Arduino Uno	Función / Tipo de Señal
Pantalla OLED SH1106	VCC	5V	Alimentación (+5V DC)
	GND	GND	Referencia de Tierra
	SCL	A5	Reloj del Bus I2C (Serial Clock)
	SDA	A4	Datos del Bus I2C (Serial Data)
Teclado Matricial 4x4	Fila 1	2	Salida Digital (Barrido FSM)
	Fila 2	3	Salida Digital (Barrido FSM)
	Fila 3	4	Salida Digital (Barrido FSM)
	Fila 4	5	Salida Digital (Barrido FSM)
	Columna 1	6	Entrada Digital (Internal Pull-Up)
	Columna 2	7	Entrada Digital (Internal Pull-Up)
	Columna 3	8	Entrada Digital (Internal Pull-Up)
	Columna 4	9	Entrada Digital (Internal Pull-Up)
LED Verde (Listo)	Ánodo (+)	10	Salida Digital HMI (Estado IDLE Vacío)
LED Rojo (Overflow)	Ánodo (+)	11	Salida Digital HMI (Alarma de Memoria)
LED Amarillo (Ingreso)	Ánodo (+)	12	Salida Digital HMI (Buffer con Datos)
LED Azul (Enviado)	Ánodo (+)	13	Salida Digital HMI (Transmisión UART)